01. JDBC Class

01. JDBC 사용 Class

01-01. DriverManager

01-01-01. DriverManager Class

- 데이터 원본(= Database)에 JDBC driver를 통하여 Connection을 만드는 역할을 한다.
- Driver란 중간에서 연결하는 역할을 의미한다.
- Class.forName() method를 통해 생성되어 메모리에 할당된다.
  - **Class**.forName()에서 **Class** 클래스는 클래스의 메타 정보 가지고 있다.
  - forName() 메소드에 풀클래스명을 작성하면 해당 클래스를 메모리에 올려 사용할 준비를 하도록 하는 것으로, 동적 로딩을 이용하여 DB driver를 로딩한다.

01-01-02. DriverManager Class 사용 시 주의 사항

1. 반드시 예외 처리를 해야 한다.
2. 직접 instance 생성이 불가하고, DriverManager 클래스의 getConnection() 메소드를 사용하여 Connection instance를 생성할 수 있다.

01-01-03. DriverManager Class 활용 예시

1. Class.forName() 메소드를 활용해 사용할 Driver를 등록한다.
   - 아래 코드는 MySQL JDBC Driver 클래스를 등록한다.
   - ClassNotFoundException에 대한 예외 처리를 반드시 해야 한다.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Drive/rManager를 이용해 Connection 객체를 생성할 수 있다.
   - getConnection() 메소드의 매개변수 속성은 다음과 같다.
     - url : 연결할 Database의 주소
     - user : Database에 접속할 user 이름
     - password : Database 접속을 위한 비밀번호

```
Connection con  = null;
try {
    con = DriverManager.getConnection("jdbc:mysql://loca
lhost/employee", "sahmyook", "sahmyook");
}
```

3. Properties 파일을 활용하여 수기로 작성하지 않고 속성 값을 불러와 코드를 작성할 수 있다.
   - 수기로 매번 설정 속성을 작성할 경우 다음과 같은 문제점이 있으므로 설정 속성 파일을 분리하여 사용하는 것이 일반적이다.

     1) 수기로 작성하면서 오타가 발생할 가능성이 높아진다.

     2) 설정 속성에 수정 사항이 발생할 경우 파일마다 번거롭게 수정해야 하므로, 유지보수 비용이 증가한다.

     3) Connection을 사용하는 파일마다 동일한 코드를 중복 작성하게 된다.
   - JDBD Configuration Properties 파일 작성

```
driver=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://localhost/employee
user=sahmyook
password=sahmyook
```

   - Properties 파일을 읽어와 Connection 객체를 생성하는 코드 작성
     - properties 파일에 key-value 방식으로 저장한 설정 정보를 읽어온다.
     - 인자로 url, user, password를 줘도 되고, url, prop를 줘도 된다. (user와 password를 분리하지 않고 보낼 수 있다.)

```
Properties prop = new Properties();
Connection con = null;

try {
    prop.load(new FileReader("jdbc-config.propertie
s"));

    String driver = prop.getProperty("driver");
    String url = prop.getProperty("url");
    String user = prop.getProperty("user");
    String password = prop.getProperty("password");

    Class.forName(driver);
    con = DriverManager.getConnection(url, user, pass
word);
}
```

01-02. Connection

01-02-01. Connection Class

- 특정 데이터 원본(= Database)과 연결된 Connection을 나타낸다.
- 쿼리문을 실행할 수 있는 Statement 혹은 PreparedStatement 객체를 생성할 수 있는 기능을 제공한다.
    - Connection instance를 생성하고 createStatement() 메소드를 호출하여 Statement instance를 생성할 수 있다.
    - 따라서 SQL문 실행 전 우선 Connection instance가 있어야 한다.

01-02-02. Connection Class 활용

- Connection 객체 생성 시 생성 전에 설정한 데이터베이스 주소로 ping을 보내 연결이 정상일 경우에 객체를 생성한다.
- Connection 객체 자원은 사용 후 반드시 반납해야 한다.

- Connection 객체를 생성하는 코드 예시는 위에서 DriverManager 클래스를 활용하여 확인하였으므로 생략한다.
- Connection 객체를 생성하는 코드 또한 중복 작성하게 되므로, Template 클래스의 static 메소드로 Connection 객체를 생성하거나 반납하는 코드를 작성하여 공통으로 사용하도록 하는 것이 일반적이다.
    - 자원 반납 시에는 null 체크와 이미 반납된 자원이 아닌지 확인해야 한다.
    - Template 코드 작성 예시

```
public class JDBCTemplate {

    public static Connection getConnection() {
        Connection con = null;
                //DriverManager를 활용한 Connnection 객체 생성 코드 생략
        return con;
    }

    public static void close(Connection con) {
        try {
            if(con != null & !con.isClosed()) {
                con.close();
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

01-03. Statement

01-03-01. Statement Class 특징

- SQL문을 저장하고 실행한 뒤 결과를 받아 반환해주는 메소드들이 묶여 있는 타입의 클래스이다.

01-03-02. Statement 객체 생성 및 사용

1. Connection class의 createStatement() 메소드를 호출하여 Statement instance를 생성한다.
2. 생성한 instance의 executeQuery() 메소드를 호출하여 SQL문 수행한다. (SQL문을 String 형태로 인자로 전달한다.)

- 사용 예시

```
try {
    String query = "SELECT ID, LAST_NAME FROM EMP";
    stmt = conn.createStatement(); // conn = Connection
객체이다.
    rset = stmt.executeQuery(query);
} catch (SQLException e) {
    e.printStackTrace();
}
```

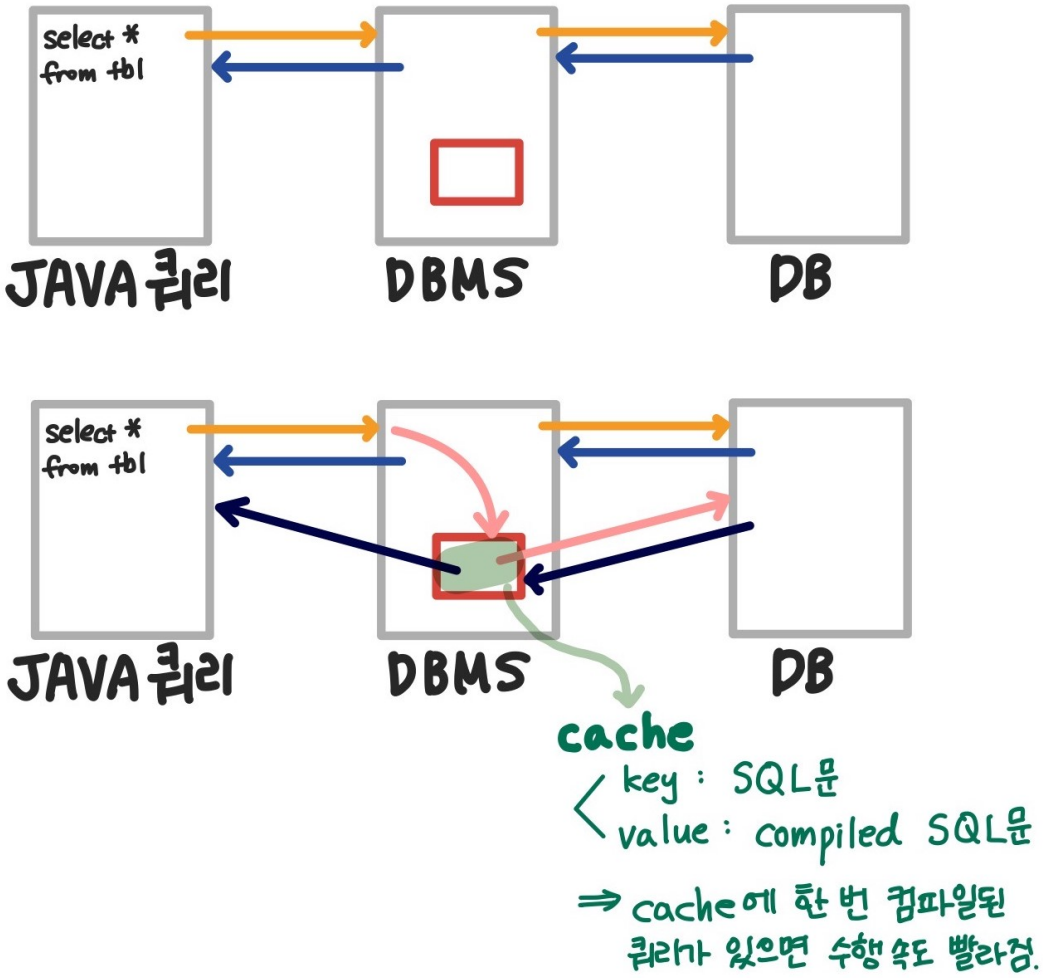
01-04. PreparedStatement

01-04-01. PreparedStatement Class 특징

- PreparedStatement도 Statement이다. 따라서 Template 클래스를 작성하여 Statement를 close() 하는 메소드를 함께 사용할 수 있다.
- 완성된 쿼리문과 미완성된 쿼리문(= 위치홀더를 사용한 쿼리문)을 모두 사용할 수 있다.
- PreparedStatement는 위치홀더(placeholder) 개념에 해당되는 인수가 많아 특정 값만 바꾸어 여러 번 실행하는 상황에 유용하다.

01-04-02. PreparedStatement 장점

1. 수행 속도가 빠르다.



- Java 쿼리문은 파싱을 통해 유효성 검사, 연산식 등의 유효성 검사를 거친 뒤에 컴파일되어 Database에서 쿼리를 수행해 결과 값을 가지고 오는 구조이다.
- Statement는 SQL문 실행 시 쿼리문을 전달하므로, 매번 새로운 쿼리로 인식하기 때문에 조건값에 따라 컴파일을 새로 해야 한다.
- 하지만 PreparedStatement는 조건값을 ?로 두고 쿼리를 준비시킨 뒤, 쿼리는 변경하지 않고 변경되지 않으므로 바인딩 되는 변수만 바꿔 조회한다. 따라서 한번 컴파일한 뒤 캐시에 담아두고 수행한다.

⇒ PreparedStatement가 Statement보다 수행 속도가 빠르다.

2. SQL injection 공격에 대하여 안전한다.
   - Statement 사용 시 전달하는 조건 변수에 **OR 1=1** 조건을 작성하면 시스템이 제공하는 의도와 다르게 데이터가 조회될 수 있다.

- 예를 들어 아래처럼 조건을 주게 되면, empName이 홍길동인 사람이 아니라 empID가 200인 사람의 정보를 조회하게 된다.

```
empName = "홍길동' OR 1=1 AND EMP_ID = '200";
```

- 하지만 PreparedStatement와 위치홀더를 사용하면, 조건으로 세팅되는 값을 메소드의 지정 형식에 따라 따옴표로 감싸는 등의 처리를 알아서 한다.
- 따라서 **OR 1=1** 조건이 악의적인 의도대로 사용되지 못하므로 Sql Injection 공격에 안전한다.

▼ (참고) SQL Injection

> ❗ **SQL Injection이란**
>
> - 악의적으로 SQL문을 실행되게 하여 Database를 비정상적으로 조작하는 코드 주입 공격이다.
> - 특히 관리자 계정으로 접근해 사용자 정보를 해킹하는 사례가 있었다.
> - 매우 초보적인 방법이라 유용한 해킹 기법 아니며, 현재는 거의 모든 사이트에서 방어하고 있다.

- 따라서 대부분의 경우PreparedStatement가 빠르고 안전하므로, 특수 케이스가 아니면 PreparedStatement 사용을 권장한다.

01-04-03. PreparedStatement 객체 생성 및 사용

1. Connection class의 preparedStatement() 메소드를 사용하여 instance 생성한다.
2. SQL문을 위치홀더(placeholder) 「?」로 표현하는 String으로 정의한다.
3. PreparedStatement의 setString() 메소드로 위치홀더의 순서와 넣어 줄 변수 값을 세팅한다.
4. 생성하고 위치홀더 값을 세팅한 instance의 executeQuery() 메소드를 호출하여 SQL문 수행한다.

- 사용 예시

```
try {
    String query = "INSERT INTO MEMBER VALUES(?, ?)";
    pstmt = conn.preparedStatement(query);
    pstmt.setString(1, id);
    pstmt.setString(2, password);
    rset = pstmt.executeQuery();
} catch (SQLException e) {
    e.printStackTrace();
}
```

01-04-04. SQL문 관리

- SQL문을 XMI파일로 분리하여 관리할 수 있다.
  - 쿼리 XMI 파일 작성 예시

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?
>
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/
properties.dtd">
<properties>
    <comment/>
    <entry key="selectEmpByFamilyName">
        SELECT
        E.*
        FROM EMPLOYEE E
        WHERE E.EMP_NAME LIKE CONCAT(?, '%')
    </entry>
</properties>
```

  - XML 파일의 쿼리를 읽어와 사용하는 코드 작성 예시

```
try {
    prop.loadFromXML(new FileInputStream("employee-qu
ery.xml"));
    String query = prop.getProperty("selectEmpByFamil
yName");
```

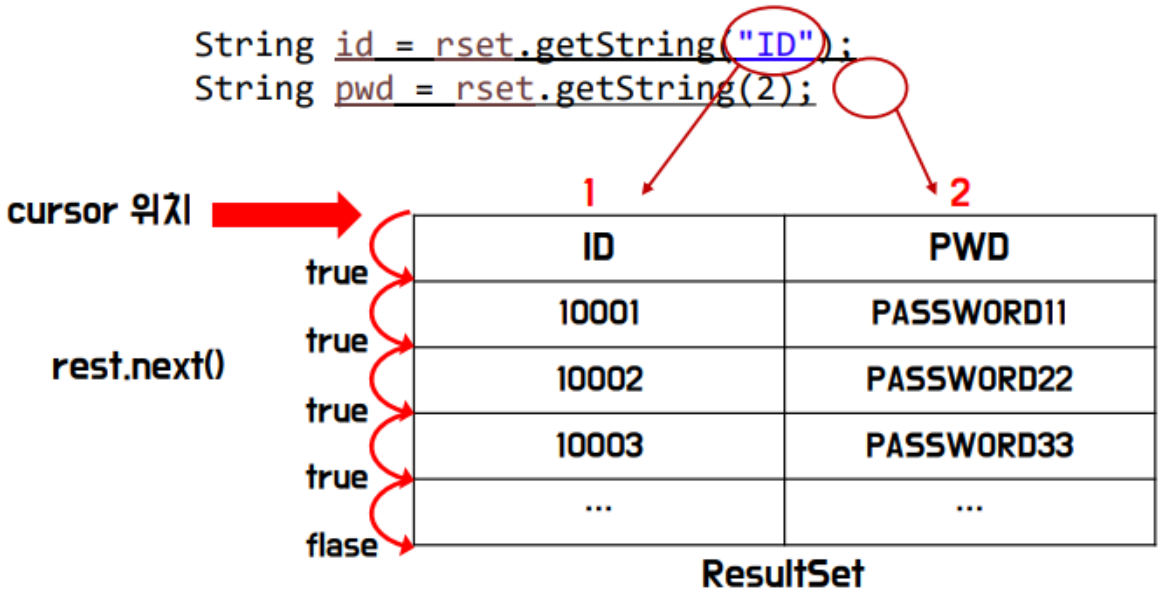
```
    pstmt = con.prepareStatement(query);
    pstmt.setString(1, empName);

    rset = pstmt.executeQuery();
}
```

01-05. ResultSet

01-05-01. ResultSet Class

- SELECT문 수행 성공 시 반환한 결과값을 받아오는 객체이다.
- SQL문에 의해 생성된 결과 테이블을 담고 있다.
- 커서(cursor)로 특정 행에 대한 참조 조작을 할 수 있다.



- Connection과 Statement도 close()로 자원 반납하듯, ResultSet도 close()를 통해 자원을 반납해야 한다.

01-05-02. ResultSet method

- getString()

    - ResultSet의 현재 커서 위치에 존재하는 로우에서 인자로 전달한 컬럼의 결과 값을 가지고 온다.
    - 데이터 자료형에 따라 **get*자료형*(" *컬럼명*")** 형식으로 사용한다.
- next()
    - ResultSet의 커서 위치를 하나 내리면서 다음 행이 존재하면 true 존재하지 않으면 false를 반환한다.
    - while문을 활용하여 쿼리 수행 결과의 마지막 행까지 반복 수행하여 결과를 한 행씩 가지고 올 때 활용한다.
    - 단, 수행 결과 반환될 컬럼이 1건임이 확실하면 while 블럭 대신 if문에 조건으로 사용할 수 있다.
- 적용 코드 예시

```
while(rset.next()) {
    System.out.println(rset.getString("EMP_ID") + ", " +
rset.getString("EMP_NAME"));
}
```

---

02. JDBC 코드 작성

02-01. JDBC 코드 작성 절차

02-01-01. JDBC 코드 작성 절차 실습

1. DriverManager에 해당 DBMS Driver를 등록한다.
    - 반드시 **ClassNotFoundException** 처리해야 한다.
    - 아래 코드는 MySql Driver를 등록한 것이다.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Driver로부터 Connection instance를 획득하여 DBMS에 연결한다.
    - 반드시 **SQLException** 처리해야 한다.

```
public static Connection getConnection(String url, Strin
g user, String password) throws SQLException {
    Connection conn = DriverManager.getConnection("jdbc:
mysql://localhost/employee", "shamyook", "shamyook");
}
```

3. Connection 객체의 메소드를 활용하여 Statement 또는 PreparedStatement instance를 생성한다.
   - PreparedStatement 객체 생성 시에는 준비시킬 쿼리를 미리 작성하여 인자로 전달해야 한다.

```
Statement stmt = conn.createStatement();
```

```
PreparedStatement pstmt = conn.preparedStatement(query);
```

4. Statement에서 제공하는 method를 사용하여 SQL문을 실행한다.
   - SELECT문은 조회 결과 테이블을 ResultSet을 반환하고, INSERT/UPDATE/DELETE문은 정상적으로 작업이 수행된 행수(로우수)를 정수형으로 반환한다.
   - SELECT문 수행 예시

```
String query = "SELECT ID, LAST_NAME FROM EMP";
ResultSet rset = stmt.executeQuery(query);

while(rset.next()){
    System.out.println(rset.getString("ID") + "\t" + rse
t.getString(2));
}
```

   - UPDATE문 수행 예시

```
String query = "UPDATE EMP SET LAST_NAME = ‘KIM’" + " WH
ERE ID = ‘10000’";
```

```
int result = stmt.executeUpdate(query);
```

5. 반환받은 결과에 따른 로직을 수행한다.
    - 예를 들어, 반환받은 결과 데이터를 객체에 담아 화면으로 반환하거나 삽입/수정/삭제 작업의 정상 여부에 따라 commit 또는 rollback을 결정하여 처리한다.

6. Connection close 및 사용한 객체를 모두 반환한다.

```
rset.close(); //ResultSet 사용한 경우 반납 처리
stmt.close(); //Statement 반납 처리
conn.close(); //Connection 반납 처리
```